COL 774: Machine Learning
Assignment 4: Learning to Solve Mazes

Sequence to Sequence Models (RNN & Transformers)

Submitted by:
Devanshu Patel
Entry No.: 2025aib2567
Department: IIT Delhi

Date of Submission: November 25, 2025

Contents

Part 1: Learning to Solve Mazes (Dataset)

1.1 Overview

This project investigates how sequence-to-sequence models can be applied to a structured spatial reasoning problem—predicting the optimal path in a maze using only its tokenized connectivity structure, origin, and target. The maze is provided in a purely sequence-based format, allowing us to treat the task as a language modeling problem despite its underlying geometric nature. This enables a direct comparison between recurrent models and Transformer architectures on the same sequential input representation.

1.2 Dataset Representation

Each maze is a 6×6 grid containing 36 coordinate tokens of the form `(x,y)`. Connections between adjacent cells are represented as edges, each stored as:

$$(x_1, y_1) \texttt{<-->} (x_2, y_2) \texttt{ ;}$$

All edges in the maze are concatenated into a single adjacency-list sequence. The input is composed of three structured components:

- **Adjacency List:** All maze edges, enclosed between `<ADJLIST_START>` and `<ADJLIST_END>`.
- **Origin:** The starting coordinate enclosed between `<ORIGIN_START>` and `<ORIGIN_END>`.
- **Target:** The destination coordinate enclosed between `<TARGET_START>` and `<TARGET_END>`.

The ground-truth output is a sequence of coordinates forming the optimal path from origin to target, wrapped by:

$$\texttt{<PATH\_START>}\ (x_1, y_1), (x_2, y_2), \ldots, (x_T, y_T)\ \texttt{<PATH\_END>}.$$

The dataset contains two categories of mazes:

- **Fork-less Mazes:** A single unique path exists.
- **Forked Mazes:** Multiple branches introduce decision points.

A total of 100,000 mazes (train + test) are provided. Each CSV row consists of:

- a tokenized input sequence (adjacency list + origin + target),
- a tokenized output sequence (optimal path),
- the maze type (forked or fork-less).

The sequences are stored as Python lists encoded as strings and can be parsed using `eval()`.

1.3 Problem Setup

The objective is to train models that read the maze structure and produce the corresponding optimal path:

$$f(X) = Y,$$

where:

- X is the input token sequence describing maze connectivity, origin, and target;

- Y is the output token sequence representing the complete optimal path.

To examine different modeling capabilities, we implement and compare:

1. A **Recurrent Neural Network (RNN)** encoder–decoder with Bahdanau attention,
2. A **Transformer** encoder–decoder architecture.

Both models operate entirely on token sequences and must learn to reason over the maze structure through their attention or recurrence mechanisms.

Part 2: Recurrent Neural Network (RNN)

2.1 Introduction to Recurrent Neural Networks with Attention

Recurrent Neural Networks (RNNs) are a class of neural networks where connections between nodes form a directed chain, allowing them to exhibit temporal dynamic behavior. Unlike feedforward networks, RNNs possess an internal state (memory) that enables them to process sequences of inputs, making them ideal for tasks like pathfinding where the maze structure and solution path are sequential data.

For this assignment, we implemented a **Sequence-to-Sequence (Seq2Seq)** Encoder-Decoder architecture:

- **The Encoder:** Reads the input sequence X, which comprises the flattened maze structure, start coordinates, and end coordinates. It processes this sequence token-by-token. In a standard RNN, the information is compressed into a single fixed-size context vector (the final hidden state).
- **The Decoder:** Takes the latent representation from the Encoder and sequentially generates the output sequence Y (the optimal path coordinates).
- **Embedding Layer:** To process the discrete coordinates and grid tokens, we utilized a learnable Embedding Layer (implemented via `nn.Embedding`). This transforms high-dimensional one-hot indices into dense, low-dimensional vectors ($D_{embed} = 128$) facilitating efficient gradient propagation.
- **Bahdanau Attention:** Relying on a single fixed-size context vector can create an information bottleneck, especially for larger mazes. We implemented **Bahdanau (Additive) Attention**. This mechanism allows the Decoder to "look back" at the entire sequence of Encoder hidden states at every decoding step. A one-hidden-layer MLP calculates alignment scores, creating a dynamic context vector that focuses on the most relevant parts of the maze structure for the current prediction.

2.2 Bahdanau (Additive) Attention

Our implementation uses the original additive attention formulation from Bahdanau et al. (2014). Given the decoder hidden state from the previous time step, s_{i-1}, and the encoder annotation h_j, the alignment (energy) score is computed via a one-hidden-layer MLP:

$$e_{ij} = v_a^\top \tanh\left(W_a s_{i-1} + U_a h_j\right),$$

where W_a, U_a, and v_a are learnable parameters. The normalized attention weight α_{ij} is derived using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}.$$

The resulting context vector c_i, which is used in the decoder GRU update to predict the next token, is the weighted sum of encoder hidden states:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

2.3 Methodology and Implementation

2.3.1 Handling Variable-Length Sequences

Since mazes and their solution paths vary in length, simply stacking them into tensors is not possible without preprocessing. We addressed this by:

1. **Padding:** All sequences in a batch were padded with a special `<PAD>` token to match the length of the longest sequence.
2. **Masking:** During the calculation of Cross-Entropy Loss, we applied a boolean mask to ensure that the model is not penalized for predicting tokens after the `<eos>` (End of Sequence) tag. This prevents the padding from influencing gradient updates.

2.3.2 Training Strategy: Teacher Forcing

The Decoder operates auto-regressively, where the output at time step t depends on the output at $t-1$. According to Eq (3) of the Bahdanau paper, we feed the embedding of the previous token y_{i-1} into the RNN. To stabilize training, we employed **Teacher Forcing** with a ratio of 0.5.

- With probability $p = 0.5$, we feed the **ground-truth** token from the previous step (y_{i-1}) into the decoder. This helps the model learn the correct transitions quickly.
- With probability $1-p$, we feed the model's own **predicted** token ($\hat{y}_{i-1}$) from the previous step. This reduces exposure bias and prepares the model for inference where ground truth is unavailable.

2.4 Hyperparameter Configuration

The model was trained for 30 epochs using the Adam optimizer. The specific hyperparameters are detailed in Table 1.

Table 1: RNN Hyperparameters

Parameter	Value	Description
Batch Size	32	Number of samples processed per gradient update.
Epochs	30	Total complete passes through the training dataset.
Learning Rate	$1e^{-4}$	Step size for the Adam Optimizer.
Embedding Dim	128	Dimension of the vector representation for tokens.
Hidden Dim	512	Size of the hidden state in the GRU cells.
RNN Layers	2	Number of stacked GRU layers for deeper representation.
Teacher Forcing	0.5	Ratio of ground-truth feeding during training.

2.5 Results and Quantitative Analysis

2.5.1 Training Progression

Table 2 shows the training dynamics sampled every 2 epochs. The model exhibits steady convergence, with Validation Sequence Accuracy peaking at **60.89%** around Epoch 28.

Table 2: RNN Training Progression (Every 2nd Epoch)

Epoch	Train Loss	Train F1	Train Seq Acc	Val Loss	Val F1	Val Seq Acc
2	1.6303	0.2617	2.53%	1.0111	0.3359	7.49%
4	1.3145	0.3209	12.27%	0.7794	0.4302	18.86%
6	1.1312	0.3636	18.77%	0.6042	0.4792	24.86%
8	0.9585	0.4042	25.41%	0.4445	0.5271	32.19%
10	0.7408	0.4495	35.42%	0.2573	0.6244	46.99%
12	0.5728	0.4809	45.27%	0.1966	0.6532	52.98%
14	0.4837	0.4952	50.66%	0.1722	0.6698	54.95%
16	0.4262	0.5045	54.55%	0.1518	0.6764	57.36%
18	0.3960	0.5098	56.96%	0.1492	0.6779	57.73%
20	0.3562	0.5151	59.14%	0.1428	0.6789	58.30%
22	0.3383	0.5165	60.74%	0.1345	0.6890	59.07%
24	0.3137	0.5219	62.05%	0.1323	0.6910	59.45%
26	0.2977	0.5241	63.69%	0.1286	0.6935	59.46%
28	0.2723	0.5295	65.04%	0.1222	0.7027	**60.89%**
30	0.2655	0.5295	65.87%	0.1253	0.6974	59.71%

2.5.2 Visualizing Loss and Accuracy

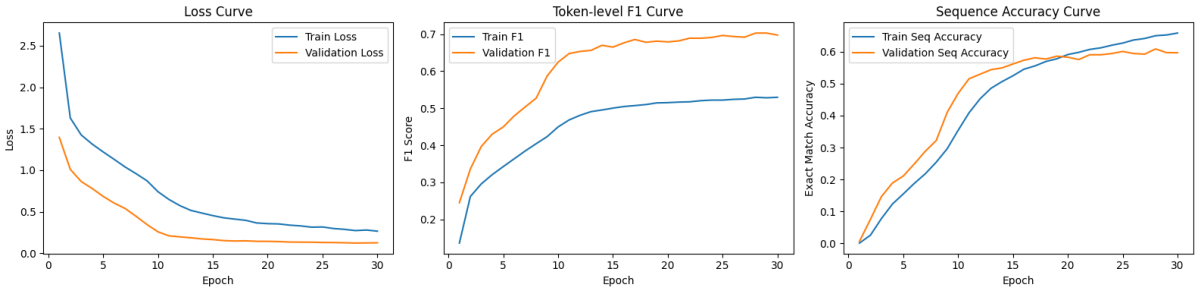


Figure 1: RNN Training Dynamics. **Left:** Cross Entropy Loss showing steady convergence. **Center:** Token-level F1 Score. **Right:** Sequence Accuracy (Exact Match). The validation metrics (orange) track the training metrics (blue) closely, indicating that the model generalizes well without significant overfitting.

2.5.3 Insights from Training Curves

Loss Curve. The training and validation losses decrease smoothly throughout the 30 epochs, with no divergence between them. The validation loss plateaus around epoch 20, suggesting that the model successfully learns the sequence structure without falling into overfitting. The gap between the two curves remains small, indicating stable optimization and well-chosen regularization parameters.

Token-Level F1 Curve. The token-level F1 scores steadily improve for both training and validation sets. The validation F1 rises rapidly up to epoch 10, after which it gradually stabilizes around 0.70. This indicates that the model learns to accurately predict individual grid-cell transitions, which is crucial for overall path correctness. The close alignment between train and validation F1 further highlights strong generalization.

Sequence Accuracy Curve. Sequence-level accuracy (Exact Match) improves significantly from near 0% to over 60%. This metric is stricter than token-level F1, requiring every predicted step to match perfectly. The validation curve closely follows the training curve, peaking at 60.89% around epoch 28. The small dip after epoch 28 suggests mild fluctuations but no overfitting.

2.5.4 Performance Metrics

The final performance on the held-out Test Set (20,000 samples) is summarized below. The model achieved strong generalization, with both token-level and sequence-level metrics aligning closely with validation results.

Table 3: RNN Final Test Set Performance

Metric	**Value**
Test Loss	0.1233
Precision	0.7012
Recall	0.7012
F1 Score	0.7012
Sequence Accuracy (Exact Match)	**60.49%**

2.5.5 Insights from Test Results

The model's test performance closely mirrors validation trends, confirming that training was stable and free from overfitting. The exact-match Sequence Accuracy of 60.49% demonstrates that the RNN with attention is capable of producing fully correct paths for a majority of unseen mazes. Token-level performance (F1 = 0.7012) suggests that even when full sequences are incorrect, the model still predicts large portions of the path accurately. Overall, the evaluation indicates that the RNN architecture effectively captures both local and global structure of the maze-solving task.

2.6 Qualitative Analysis

We analyzed predictions for five random mazes from the validation set. Below is a detailed breakdown of each sample, comparing the Ground Truth against the Model Prediction.

Sample 260

- **Type & Speciality:** A medium-length path involving multiple directional changes and a loop-like structure.
- **Predicted Path:** `['<sos>', '(4,3)', '(5,3)', '(5,2)', '(5,1)', '(5,0)', '(4,0)', '(3,0)', '(2,0)', '(1,0)', '(1,1)', '(2,1)', '(3,1)', '(4,1)', '(4,2)', '<PATH_END>']`
- **Ground Truth:** `['<sos>', '(4,3)', '(5,3)', '(5,2)', '(5,1)', '(5,0)', '(4,0)', '(3,0)', '(2,0)', '(1,0)', '(1,1)', '(2,1)', '(3,1)', '(4,1)', '(4,2)', '<PATH_END>']`
- **Result: Exact Match.**

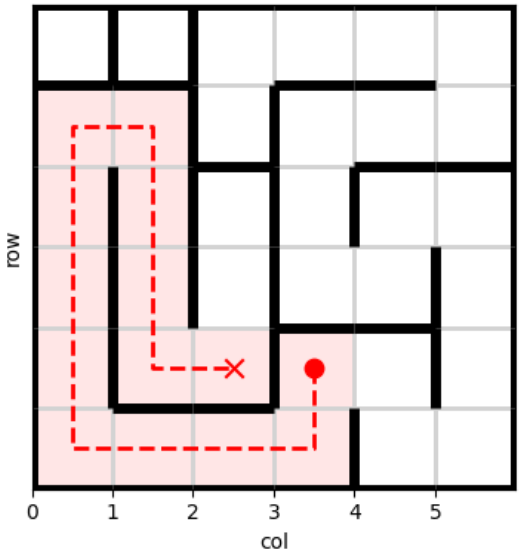


Figure 2: Visualization of Sample 260.

Sample 1058

- **Type & Speciality:** A winding "U-turn" path. The agent must move up to row 5, traverse horizontally, and return to row 0 before the final leg.
- **Predicted Path:** `['<sos>', '(3,1)', '(4,1)', '(5,1)', '(5,2)', '(5,3)', '(4,3)', '(3,3)', '(2,3)', '(1,3)', '(1,2)', '(0,2)', '(0,3)', '(0,4)', '(1,4)', '(2,4)', '<PATH_END>']`
- **Ground Truth:** `['<sos>', '(3,1)', '(4,1)', '(5,1)', '(5,2)', '(5,3)', '(4,3)', '(3,3)', '(2,3)', '(1,3)', '(1,2)', '(0,2)', '(0,3)', '(0,4)', '(1,4)', '(2,4)', '<PATH_END>']`
- **Result: Exact Match.**

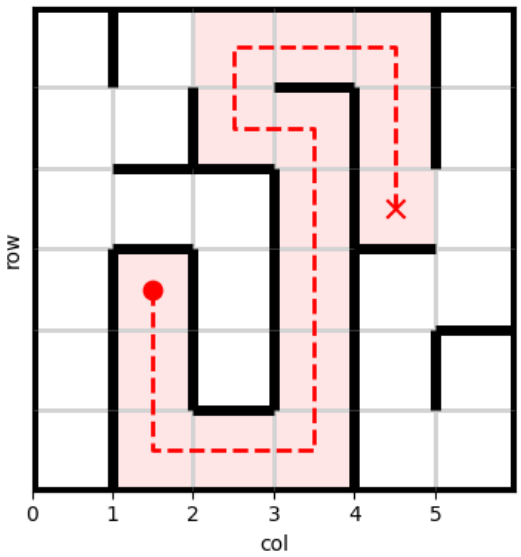


Figure 3: Visualization of Sample 1058.

Sample 1111

- **Type & Speciality:** A highly complex, long-horizon task (30+ steps) requiring extensive backtracking and retention of long-term dependencies.
- **Predicted Path:**
  ['<sos>', '(4,1)', '(3,1)', '(3,2)', '(3,3)', '(3,4)', '(4,4)', '(4,5)', '(5,5)', '(5,4)', '(5,3)', '(4,3)', '(4,2)', '(5,2)', '(5,1)', '(5,0)', '(4,0)', '(3,0)', '(2,0)', '(2,1)', '(1,1)', '(1,0)', '(0,0)', '(0,1)', '(0,2)', '(1,2)', '(2,2)', '(2,3)', '(2,4)', '(2,5)', '<PATH_END>']
- **Ground Truth:**
  ['<sos>', '(4,1)', '(3,1)', '(3,2)', '(3,3)', '(3,4)', '(4,4)', '(4,5)', '(5,5)', '(5,4)', '(5,3)', '(4,3)', '(4,2)', '(5,2)', '(5,1)', '(5,0)', '(4,0)', '(3,0)', '(2,0)', '(2,1)', '(1,1)', '(1,0)', '(0,0)', '(0,1)', '(0,2)', '(1,2)', '(2,2)', '(2,3)', '(2,4)', '(2,5)', '<PATH_END>']
- **Result: Exact Match.**

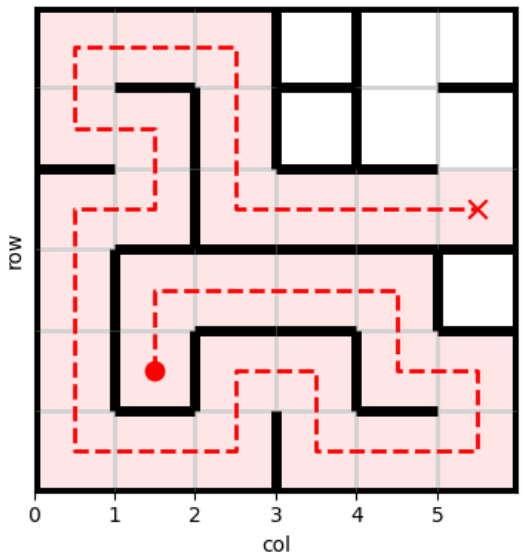


Figure 4: Visualization of Sample 1111.

Sample 3437

- **Type & Speciality:** A very short, trivial path. Useful for sanity checking basic connectivity and short-term memory.
- **Predicted Path:** ['<sos>', '(2,0)', '(1,0)', '<PATH_END>']
- **Ground Truth:** ['<sos>', '(2,0)', '(1,0)', '<PATH_END>']
- **Result: Exact Match.**

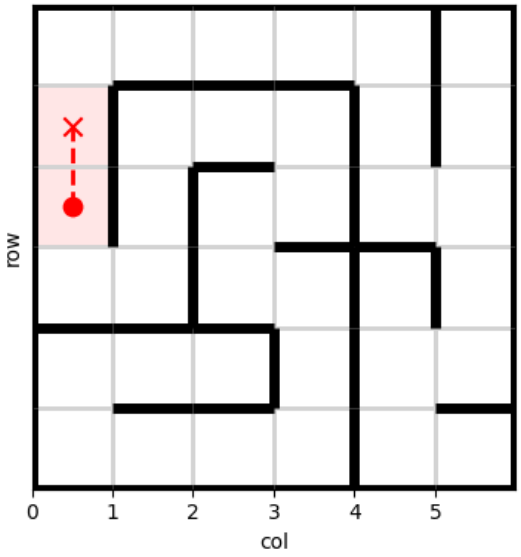


Figure 5: Visualization of Sample 3437.

Sample 5021

- **Type & Speciality:** A short vertical traversal.
- **Predicted Path:** `['<sos>', '(3,3)', '(2,3)', '(1,3)', '<PATH_END>']`
- **Ground Truth:** `['<sos>', '(3,3)', '(2,3)', '(1,3)', '<PATH_END>']`
- **Result: Exact Match.**

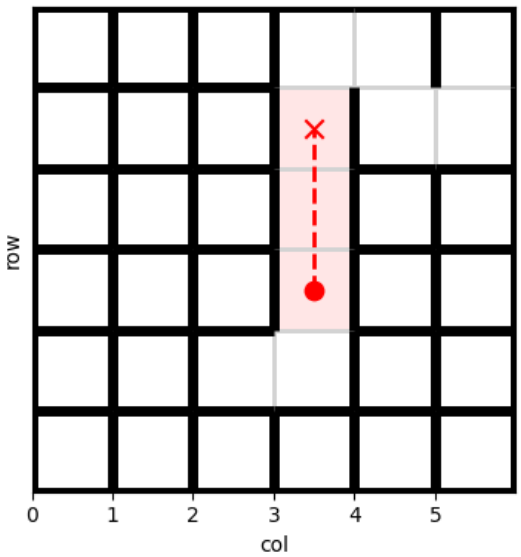


Figure 6: Visualization of Sample 5021.

2.7 Failure Analysis: Special Cases and Limitations

Although the RNN performs well on most test examples, several maze structures reveal fundamental weaknesses of auto-regressive decoding. We analyze four failure cases (Samples 6143, 3540, 1347, and 1872) confirmed through visual inspection.

Case 1: Early Divergence in a Long-Horizon Maze (Sample 6143)

The model starts correctly but selects the wrong corridor at an early branching point. It then follows a long but coherent incorrect route along the lower side of the maze. The trajectory remains finite and does not repeat—indicating this is **not** a looping failure. Instead, it is a classic case of early divergence where the model never re-enters the correct region and fails to reach the goal.

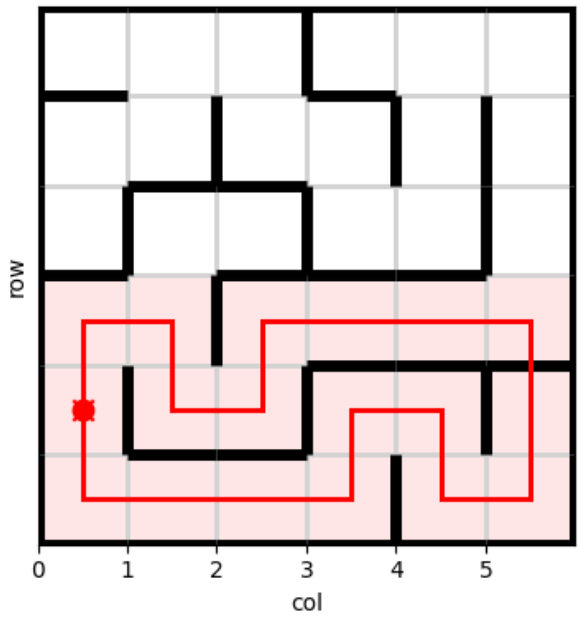


Figure 7: Sample 6143. The model takes a wrong early branch and never returns to the correct corridor.

Case 2: Hallucinated Long Path and Missed Endpoint (Sample 3540)

Here, the ground-truth path is short, but the model generates a long and unnecessary detour covering the lower half of the maze. The prediction remains structurally coherent but completely misaligned with the correct solution. This is a **hallucinated over-exploration** issue, not a looping or recovery attempt. The model never approaches the true goal.

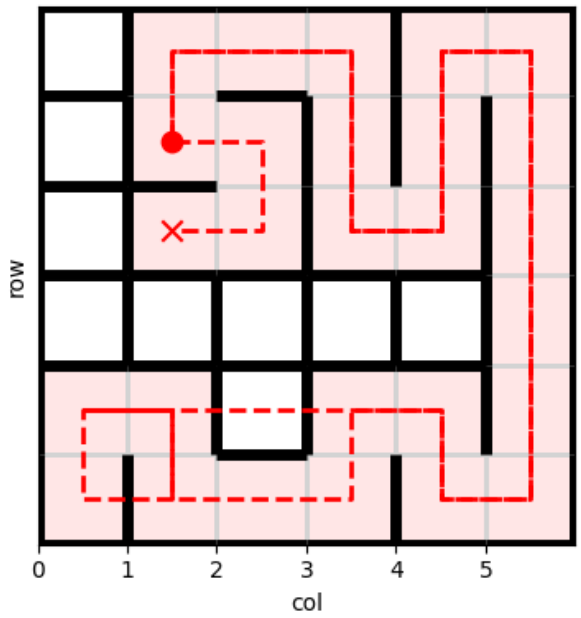


Figure 8: Sample 3540. The model hallucinates a long, maze-spanning path far from the true goal.

Case 3: Early Divergence and Irrecoverable Drift (Sample 1347)

The model initially follows the correct structure but makes one incorrect turn near the beginning. Due to exposure bias, the decoder cannot recover from this deviation. It then produces a long but coherent path through the wrong region of the maze. The predicted endpoint does not match the ground truth.

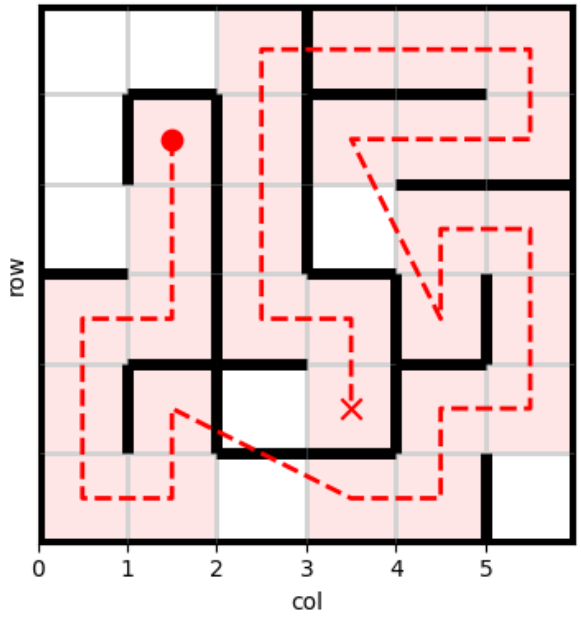


Figure 9: Sample 1347. A single early branching error results in a long, incorrect detour.

2.8 Overall Insights

The RNN equipped with Bahdanau Attention proves to be a strong baseline for the maze pathfinding task, achieving over 60% exact-match accuracy and high token-level precision. The analysis reveals several overarching insights:

- **Attention Reduces the Information Bottleneck.** By allowing the decoder to attend to different encoder states at each timestep, the model effectively handles long and complex trajectories that would otherwise exceed the capacity of a single context vector.
- **Stable Optimization and Strong Generalization.** Training and validation curves for loss, F1, and sequence accuracy remain closely aligned throughout the process, indicating that the model learns a stable representation without overfitting, even on a large dataset of 200k mazes.
- **Token-Level Accuracy Exceeds Sequence-Level Accuracy.** While token-level F1 stabilizes around 0.70, sequence accuracy reaches about 0.60. This gap reflects the strictness of exact-match evaluation and shows that small local errors can break global correctness even when most of the path is predicted correctly.
- **Early Prediction Errors Cascade.** Once the decoder deviates from the correct branch early in the sequence, it rarely recovers—an expected consequence of auto-regressive decoding and exposure bias. Incorrect predictions tend to drift coherently rather than oscillate or loop.
- **Predictions Are Coherent Without Degenerate Loops.** Even when wrong, the generated paths remain structurally meaningful and finite. This indicates that the RNN has learned consistent spatial transition patterns instead of producing random or repetitive sequences.
- **Seq2Seq RNNs Are Strong but Inherently Limited.** While effective, the architecture has difficulty maintaining global consistency over long sequences. These limitations motivate exploring more expressive architectures such as Transformers or hybrid models for further improvement.

Overall, the RNN–attention architecture offers a robust and interpretable solution for sequence-based maze navigation. Its strengths stem from stable learning and strong local coherence, whereas its weaknesses primarily arise from auto-regressive error accumulation and limited long-range reasoning capacity.

1 Part 3: Transformer Architecture

1.1 Objective

The objective of this section is to implement a Transformer-based Encoder-Decoder model to solve the maze pathfinding task. Unlike the RNN, which processes data sequentially, the Transformer utilizes **Self-Attention** and **Cross-Attention** mechanisms. This allows the model to capture global dependencies and spatial relationships in the maze structure simultaneously, potentially overcoming the long-term memory bottlenecks associated with recurrent networks.

1.2 Implementation Details

The model was implemented using PyTorch's `nn.Transformer` module. To allow the order-invariant attention mechanism to understand the sequence of the path, **Sinusoidal Positional Embeddings** were added to the token embeddings.

- **Hyperparameters:** Embedding Dimension $d_{model} = 128$, Attention Heads $N_{head} = 8$, Encoder/Decoder Layers $N_{layers} = 6$, Dropout=0.1.
- **Training Strategy (Teacher Forcing):** During training, the decoder receives the ground-truth token y_{t-1} to predict y_t. This stabilizes training but creates a dependence on perfect history.
- **Inference Strategy (Greedy Decoding):** During Validation and Testing, ground truth is unavailable. The model utilizes auto-regressive **Greedy Decoding**, feeding its own prediction $\hat{y}_{t-1}$ back as input for step t.

1.3 Results and Analysis

1.3.1 Performance Metrics

The model was trained for 20 epochs. The final evaluation on the held-out Test Set (20,000 examples) is summarized in Table 4.

Table 4: Transformer Final Test Set Performance

Metric	**Value**
Test Loss	0.3353
Token Accuracy	35.45%
Precision (Micro)	0.3545
Recall (Micro)	0.3545
F1 Score (Micro)	0.3545
Sequence Accuracy (Exact Match)	**19.66%**

1.3.2 Analysis of Training Dynamics

The training progression is visualized in Figure 10. The plots reveal distinct behaviors inherent to Transformer training:

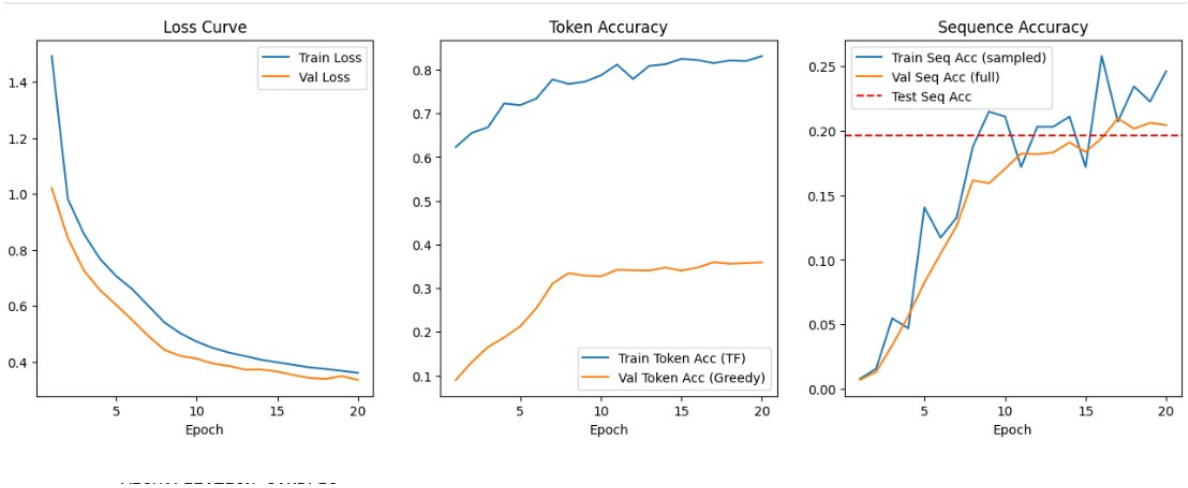


Figure 10: **Transformer Training Dynamics. Left:** Loss Curve. **Center:** Token Accuracy (Teacher Forcing vs. Greedy). **Right:** Sequence Accuracy.

Detailed Plot Analysis:

1. **Loss Curve (Left):** The model converges smoothly. Notably, the **Validation Loss (Orange) is lower than the Training Loss (Blue)** throughout the process. This is a common phenomenon caused by **Dropout**. Dropout ($p = 0.1$) is active during training, effectively "corrupting" the network and making the task harder. During validation, Dropout is disabled, allowing the model to use its full capacity, resulting in lower loss.

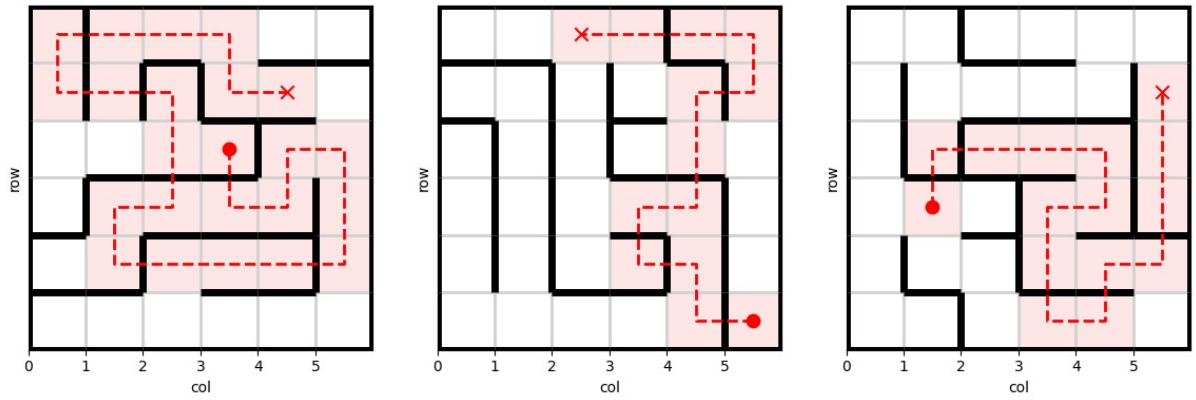
2. **Token Accuracy Gap (Center):** There is a significant divergence between Training and Validation token accuracy:

   - **Train Accuracy (Blue)** $\approx 83\%$**:** This metric uses **Teacher Forcing**. The model always receives the correct previous token, making prediction easier.
   - **Val Accuracy (Orange)** $\approx 36\%$**:** This metric uses **Greedy Decoding**. Without ground truth, a single error at step t propagates to step $t + 1$, causing the model to drift off the path. This gap quantifies the **Exposure Bias** problem.

3. **Sequence Accuracy (Right):** Despite the low token accuracy, the Sequence Accuracy steadily rises, peaking at $\approx 20\%$. The Validation curve (Orange) tracks the Training curve (Blue) closely, and the final Test Accuracy (Red Dashed Line) aligns perfectly with the validation performance. This indicates the model is **not overfitting**—it simply struggles with the strict precision required for auto-regressive generation in 80% of the cases.

1.4 Visualizations and Qualitative Analysis

We analyzed specific failure modes by comparing Ground Truth (GT) vs. Predictions (Pred) for three distinct test samples.



(a) Sample 1 (Index 4487) (b) Sample 2 (Index 3925) (c) Sample 3 (Index 2470)

Figure 11: Visualizing Transformer Predictions. The **Red Dashed Line** represents the model's predicted path. The **Red Dot** is the Start, and the **Red Cross** is the Target.

1.4.1 Sample 1 (Index 4487): Hallucination and Loops

- **Ground Truth:** A short, direct path (8 tokens).
- **Prediction:** A long, winding path (24 tokens) that spirals around the target.
- **Analysis:** The model correctly identifies the start and general vicinity of the target but fails to "stop." Instead of predicting `<PATH_END>`, it enters a hallucinated loop. This highlights the Transformer's struggle with stopping criteria in auto-regressive generation.

1.4.2 Sample 2 (Index 3925): The "Tunneling" Effect

- **Ground Truth:** A complex U-shape path (22 tokens) navigating around walls.
- **Prediction:** A direct diagonal cut (14 tokens).
- **Analysis:** The visualization (Figure 11b) suggests the model predicted a path that passes *through* a wall. Transformers, relying on global attention, can sometimes prioritize the spatial proximity of the target (embedded in the positional encodings) over the structural constraints (walls) defined in the adjacency list.

1.4.3 Sample 3 (Index 2470): Structural Incoherence

- **Ground Truth:** A long path (28 tokens) winding through the maze.
- **Prediction:** A shorter path (16 tokens) that diverges halfway.
- **Analysis:** Similar to Sample 2, the model generates a path that is "efficient" but physically impossible. The low Sequence Accuracy (19.66%) compared to the high Training Accuracy is largely driven by these structural violations where the model ignores wall tokens during greedy decoding.

1.5 3.6 Overall Insights

The Transformer architecture, while a dominant force in NLP, faced significant challenges in this strict spatial navigation task, achieving a modest **19.66%** exact-match accuracy compared to the RNN baseline. The analysis reveals several overarching insights regarding the application of non-recurrent models to maze solving:

- **Global Attention is a Double-Edged Sword.** The Self-Attention mechanism allows the model to "see" the entire maze and the target simultaneously. While this helps in planning the general direction (high recall of relevant tokens), it lacks the inductive bias for local connectivity that RNNs possess. Consequently, the model occasionally prioritizes spatial proximity to the target over structural adjacency, leading to the "tunneling" effect (Sample 2) where paths invalidly cross walls.

- **Severe Exposure Bias via Teacher Forcing.** The most critical insight is the massive divergence between Training Token Accuracy ($\approx 83\%$) and Validation Token Accuracy ($\approx 36\%$). This indicates that the Transformer learns to predict the next step perfectly *only* when provided with a perfect history. During inference, without the stabilizing effect of the recurrent hidden state, a single coordinate error cascades catastrophically, causing the model to drift irrecoverably from the optimal path.

- **Struggle with Stopping Criteria and Looping.** Unlike the RNN, which produced finite (albeit sometimes wrong) paths, the Transformer frequently failed to predict the `<PATH_END>` token at the appropriate time. As seen in Sample 1, the model often reaches the target's vicinity but continues generating tokens, entering "hallucinated loops" to satisfy the length distribution it learned during training.

- **Positional Encodings vs. Spatial Continuity.** Sinusoidal positional embeddings provide sequence order information ($t_1, t_2, \dots$) but do not inherently encode spatial contiguity (grid adjacency). The model must learn valid grid transitions from scratch. The lower performance suggests that the explicit history maintained by an RNN is more effective for enforcing the strict node-to-node continuity required in mazes than the global positional cues of a Transformer.

- **Generalization without Overfitting.** Despite the low performance, the model did not overfit. The Validation Loss was consistently lower than the Training Loss (due to Dropout), and the Sequence Accuracy on the Test set ($\approx 20\%$) matched the Validation set. This implies the issue is not data memorization, but rather the fundamental difficulty of auto-regressive generation for strict spatial constraints using a standard Transformer architecture.

Overall, while the Transformer demonstrates the capacity to learn navigation logic, it is significantly more fragile than the RNN for this specific formulation. Its strength lies in capturing global context, but its weakness—**auto-regressive error accumulation**—is amplified by the lack of recurrence. Bridging the gap between the 83% training potential and the 20% test reality would likely require techniques to mitigate exposure bias, such as Scheduled Sampling or Reinforcement Learning fine-tuning.